

VS Code + MinGW-w64 GCC : C 语言环境配置指南 (Windows)

适用对象：第一次接触 C 语言程序设计的本科一年级同学
本文以当前课程工作文件夹 `D:\code\c` 为例。
目标：能够在 VS Code 中编辑、编译、运行和调试一个 `.c` 程序。

1. 先认识三个不同的工具

安装 VS Code 并不等于安装了 C 语言编译器。

工具	作用	本课程采用
VS Code	编辑源代码	Visual Studio Code
C/C++ 扩展	提供代码补全、错误提示和调试入口	Microsoft C/C++
编译器和调试器	把 <code>.c</code> 文件编译为 <code>.exe</code> ，并支持断点调试	MinGW-w64 GCC + GDB

可以把它理解为：VS Code 是“书桌”，C/C++ 扩展是“辅助工具”，GCC 才是把源代码变成可执行程序“编译器”。

2. 安装 VS Code 和 C/C++ 扩展

1. 从 [VS Code 官方网站](#) 下载并安装 Windows 版 VS Code。
2. 启动 VS Code，按 `Ctrl+Shift+X` 打开“扩展”。
3. 搜索 `C/C++`。
4. 安装发布者为 Microsoft、扩展 ID 为 `ms-vscode.cpptools` 的扩展。

本课程不依赖 Code Runner 扩展。初学阶段建议使用本文的 GCC 编译任务，避免不同扩展的“运行”按钮造成混淆。

3. 安装 MinGW-w64 GCC

以下两种方法任选一种。不要把两套编译器路径混在同一份配置中。

方法 A：使用课程提供的 MinGW-w64 工具包

当前示例电脑使用的目录是：

```
E:\mingw64\bin\gcc.exe  
E:\mingw64\bin\gdb.exe
```

如果教师提供了同样的 `mingw64` 文件夹，请将其放到 `E:\mingw64`。确认 `E:\mingw64\bin` 中至少能找到：

```
gcc.exe  
gdb.exe
```

如果电脑没有 `E:` 盘，可以放到其他位置，但第 6 节三份 JSON 配置中的路径也必须一起修改。例如，工具包放在 `D:\software\mingw64` 时，应使用：

```
D:\software\mingw64\bin\gcc.exe  
D:\software\mingw64\bin\gdb.exe
```

方法 B：通过 MSYS2 安装（官方推荐途径）

VS Code 官方 MinGW 教程推荐通过 MSYS2 获取较新的 MinGW-w64 工具链。

1. 从 [MSYS2 官方网站](#) 下载安装程序，并按默认位置安装。
2. 安装结束后打开 MSYS2 UCRT64 终端。
3. 执行：

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

4. 出现软件包选择时按 `Enter` 接受默认选择；询问是否继续时输入 `Y`。
5. 默认的编译器目录通常是：

```
C:\msys64\ucrt64\bin
```

6. 后续配置中的编译器和调试器路径应分别写成：

```
C:\msys64\ucrt64\bin\gcc.exe  
C:\msys64\ucrt64\bin\gdb.exe
```

MSYS2 官方建议不确定环境时选择 UCRT64。它仍然使用 MinGW-w64 GCC，只是默认目录不是示例电脑上的 `E:\mingw64`。

4. 把编译器目录加入 PATH

PATH 让 Windows 和 VS Code 能够只通过名称找到 `gcc`、`gdb` 等程序。

1. 按 Windows 键，搜索“环境变量”。
2. 打开“编辑账户的环境变量”。
3. 在“用户变量”中双击 `Path`。
4. 单击“新建”，填入实际的 `bin` 目录。例如二选一：

```
E:\mingw64\bin
```

或：

```
C:\msys64\ucrt64\bin
```

5. 连续单击“确定”保存。
6. **关闭所有已经打开的 VS Code 和终端窗口，再重新打开。**旧窗口不会自动获得新的 PATH。

检查安装是否成功

在一个新打开的 PowerShell、命令提示符或 VS Code 终端中依次输入：

```
gcc --version
gdb --version
where.exe gcc
where.exe gdb
```

正确现象：前两条命令显示版本，后两条命令显示实际的 `.exe` 路径。

如果提示“无法将 `gcc` 识别为命令”或“不是内部或外部命令”，先不要继续配置 VS Code，应回到本节检查 PATH。

5. 认识本课程的示例工作文件夹

解压双系统包后，请在 VS Code 中直接打开 `windows` 文件夹；也可以将整个 `windows` 文件夹复制并重命名为 `c`，放到下面的示例位置。不要打开包含两个系统文件夹的上一层目录。

当前工作文件夹为：

```
D:\code\c
├── .vscode
│   ├── c_cpp_properties.json
│   ├── launch.json
│   ├── settings.json
│   └── tasks.json
├── C1
│   └── A.c
```

各部分含义如下：

- `D:\code\c`：工作区根目录。打开 VS Code 时应打开这个文件夹，而不是只打开某个 `A.c` 文件。

- `.vscode`：只对当前工作区生效的配置目录。
- `tasks.json`：告诉 VS Code 如何调用 GCC 编译当前 C 文件。
- `launch.json`：告诉 VS Code 如何运行或使用 GDB 调试编译出来的程序。
- `c_cpp_properties.json`：告诉 C/C++ 扩展编译器位置、C 标准和代码补全设置。
- `settings.json`：保存该工作区的编辑器设置。
- `C1\A.c`：本次上机的 C 源代码。

请在 VS Code 中选择“文件 → 打开文件夹”，打开 `D:\code\c`。左侧资源管理器应同时看到 `.vscode` 和 `C1`。

`.vscode` 是工作区配置，所以它可以服务于 `C1` 以及将来建立的其他上机子文件夹。

6. 配置当前工作区

下面的配置采用当前示例电脑的 `E:\mingw64`。如果你的实际路径不同，请使用“查找和替换”，把所有 `E:\mingw64` 换为自己的路径。

JSON 字符串中的 Windows 路径使用两个反斜杠，例如实际路径 `E:\mingw64\bin\gcc.exe` 在 JSON 中写作 `E:\\mingw64\\bin\\gcc.exe`。

6.1 `.vscode/c_cpp_properties.json`

```
{
  "configurations": [
    {
      "name": "MinGW x64",
      "includePath": ["${workspaceFolder}/**"],
      "defines": [],
      "compilerPath": "E:\\mingw64\\bin\\gcc.exe",
      "cStandard": "c11",
      "cppStandard": "c++17",
      "intelliSenseMode": "windows-gcc-x64"
    }
  ],
  "version": 4
}
```

关键项：

- `compilerPath`：GCC 的实际位置。
- `cStandard: "c11"`：本课程按 C11 标准检查代码。
- `includePath`：当前工作区内头文件的搜索范围；标准库头文件路径会由扩展向 GCC 查询，不需要手工填写。

6.2 .vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C: build release",
      "command": "E:\\mingw64\\bin\\gcc.exe",
      "args": [
        "-std=c11", "-Wall", "-Wextra", "-pedantic", "-O2", "-fdiagnostics-color=always", "${file}",
        "-o", "${fileDirname}\\${fileBasenameNoExtension}.exe"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "使用 GCC 以 C11 模式编译当前 C 文件（发布版）"
    },
    {
      "type": "cppbuild",
      "label": "C: build debug",
      "command": "E:\\mingw64\\bin\\gcc.exe",
      "args": [
        "-std=c11", "-Wall", "-Wextra", "-pedantic", "-O0", "-g3", "-fdiagnostics-color=always",
        "${file}", "-o", "${fileDirname}\\${fileBasenameNoExtension}.exe"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": false
      },
      "detail": "使用 GCC 以 C11 模式编译当前 C 文件（调试版）"
    }
  ]
}
```

几个常见编译参数：

参数	含义
-std=c11	使用 C11 标准
-Wall -Wextra -pedantic	尽量显示有教学价值的警告
-O2	优化发布版程序
-O0	调试时关闭优化，使单步执行更直观
-g3	把调试信息写入程序，供 GDB 使用
-o ...	指定输出的.exe 文件名和位置

`${file}` 表示当前编辑器中正在打开的文件。因此编译前应先单击 [A.c](#)，使它成为当前文件。

6.3 .vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "运行当前 C 文件",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "E:\\mingw64\\bin\\gdb.exe",
      "preLaunchTask": "C: build release"
    },
    {
      "name": "调试当前 C 文件",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "E:\\mingw64\\bin\\gdb.exe",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for GDB",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ],
      "preLaunchTask": "C: build debug"
    }
  ]
}
```

注意：preLaunchTask 的文字必须与 tasks.json 中对应任务的 label 完全相同。

6.4 .vscode/settings.json (可选)

```
{
  "files.defaultLanguage": "c",
  "editor.formatOnType": true,
  "editor.snippetSuggestions": "top",
  "files.encoding": "utf8"
}
```

7. 编写第一个程序

当前 `C1\A.c` 已有一个最小的 C 程序框架。为了看到明显的运行结果，可以暂时写成：

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

按 `Ctrl+S` 保存。

这段程序在做什么

- `#include <stdio.h>`：使用标准输入输出库。
- `int main(void)`：程序从 `main` 函数开始执行。
- `printf(...)`：在屏幕上输出文字。
- `\n`：换行。
- `return 0;`：告诉操作系统程序正常结束。

8. 编译和运行

方式一：先在终端中操作（建议第一次课先练习）

在 VS Code 中按 `Ctrl+`` 打开终端，然后执行：

```
cd ".\C1"
gcc A.c -std=c11 -Wall -Wextra -pedantic -o A.exe
.\A.exe
```

如果看到：

```
Hello, C!
```

说明“源代码 → GCC 编译 → EXE 运行”这一完整流程已经成功。

其中：

```
A.c  --GCC 编译-->  A.exe  --Windows 运行-->  输出结果
```

修改源代码后，必须重新编译，旧的 `A.exe` 不会自动变化。

方式二：按 **Ctrl+F5** 编译并运行（推荐日常使用）

1. 在编辑器中单击打开 **A.c**，使它成为当前文件。
2. 按 **Ctrl+F5**，即“运行但不调试”。
3. 如果第一次运行时出现配置选择，请选择“运行当前 C 文件”。
4. VS Code 会先调用 GCC 编译 **A.c**，生成 **A.exe**，然后自动运行它。
5. 在弹出的控制台中查看程序输出；如果程序要求输入，就在该控制台中输入。

简单原理如下：

```
Ctrl+F5
↓
读取 launch.json 中的“运行当前 C 文件”
↓
根据 preLaunchTask 调用 tasks.json 中的“C: build release”
↓
GCC 把当前 A.c 编译成 A.exe
↓
VS Code 直接运行 A.exe，但不启动断点调试
```

Ctrl+F5 适合只想查看程序运行结果的情况；**F5** 则会启动 GDB，适合设置断点、单步执行和观察变量。修改源代码后再次按 **Ctrl+F5**，编译任务会先重新生成最新的 **.exe**。

判断编译是否成功，应看终端中的 GCC 输出，不应只看代码下方有没有红色波浪线。如果编译失败，程序不会进入运行阶段，应先修改终端中报告的第一个错误。

9. 断点调试

调试可以暂停程序，逐行观察变量变化。

1. 单击打开 **A.c**。
2. 在某一行左侧行号栏单击，出现红点，这就是断点。
3. 打开左侧“运行和调试”（**Ctrl+Shift+D**）。
4. 在顶部选择“调试当前 C 文件”。
5. 按 **F5**。VS Code 会先执行 **C: build debug**，再由 GDB 启动程序。
6. 程序停在断点后，可使用：
 - **F10**：单步跳过（执行当前行，不进入函数内部）；
 - **F11**：单步进入函数；
 - **Shift+F11**：跳出当前函数；
 - **F5**：继续运行到下一个断点。

如果程序需要用 **scanf** 输入，请在弹出的外部控制台中输入。

10. 常见问题排查

问题 1：终端找不到 gcc

依次检查：

```
where.exe gcc
gcc --version
```

如果没有结果：

1. 检查 PATH 中填写的是包含 `gcc.exe` 的 `bin` 目录，而不是上一层目录；
2. 检查路径是否拼错；
3. 完全关闭并重新打开 VS Code；
4. 检查是否把工具包解压成了 `E:\mingw64\mingw64\bin` 这样的双层目录。

问题 2：`#include <stdio.h>` 下方出现红色波浪线

1. 先确认终端中的 `gcc --version` 成功；
2. 检查 `c_cpp_properties.json` 的 `compilerPath`；
3. 按 `Ctrl+Shift+P`，运行 `C/C++: Select IntelliSense Configuration`，选择实际的 `gcc.exe`；
4. 再运行 `Developer: Reload Window` 重新载入窗口。

红色波浪线来自 IntelliSense，真正的编译结果来自 GCC；两者配置相关，但不是同一个程序。

问题 3：提示 `miDebuggerPath is invalid` 或找不到 GDB

检查：

```
where.exe gdb
gdb --version
```

然后确认 `launch.json` 中的 `miDebuggerPath` 指向真实存在的 `gdb.exe`。使用 MSYS2 时，如果 GCC 存在但 GDB 不存在，应确认完整工具链已经安装。

问题 4：提示找不到 `preLaunchTask`

检查 `launch.json` 的 `preLaunchTask` 和 `tasks.json` 的 `label` 是否逐字相同，并确认 `tasks.json` 是有效 JSON。

问题 5：按 `F5` 后提示找不到 `.exe`

通常是编译步骤已经失败。向上查看终端中最早出现的 GCC 错误，先修改源代码，再重新按 `F5`。

问题 6：程序一闪而过

不要通过文件资源管理器双击 `.exe`。从 VS Code 终端运行 `.\A.exe`，或者使用本文的调试配置。

问题 7：中文目录导致问题吗

本教程使用英文子目录 `C1`。若工具链在中文目录中出现编码或路径错误，可把整个工作区复制到纯英文路径后重试，例如 `D:/code/c/C1/A.c`。

问题 8：修改了一个文件，却编译了另一个文件

本教程的任务使用 `${file}`，只编译当前活动文件。按 `Ctrl+Shift+B` 或 `F5` 前，先单击要编译的 `.c` 文件标签页。

问题 9：一个程序有多个 `.c` 文件怎么办

本教程的配置适合“一个 `.c` 文件就是一个程序”的入门阶段。以后学习多文件程序时，需要在 GCC 命令中列出所有源文件，或使用 Make/CMake；不要简单地给每个文件都写一个 `main` 函数。

11. 课前自检清单

上课前请逐项确认：

- ☐ VS Code 可以正常启动；
- ☐ 已安装 Microsoft C/C++ 扩展；
- ☐ `gcc --version` 有版本输出；
- ☐ `gdb --version` 有版本输出；
- ☐ 使用 VS Code 打开的是课程文件夹，而不是单独的 `.c` 文件；
- ☐ `.vscode` 中的编译器路径与自己的电脑一致；
- ☐ `Ctrl+Shift+B` 能生成 `.exe`；
- ☐ 终端可以运行 `.exe` 并看到输出；
- ☐ `F5` 可以在断点处暂停。

完成以上项目，C 语言实验环境就配置好了。

参考资料

- [VS Code 官方：Using GCC with MinGW](#)
- [VS Code 官方：Configure C/C++ IntelliSense](#)
- [MSYS2 官方：安装与 GCC](#)
- [MSYS2 官方：不同编译环境的说明](#)